Morse Code Transmitter

Jokobus

May 11, 2026

Project Overview

- **Goal:** Visual Morse code transmitter with lighthouse effect
- **Hardware:** Arduino Nano (ATmega328P) + WS2812B LED strip
- **Key Features:**
  - Interrupt-driven timing for precision and free main loop
  - Addressable RGB LEDs with wave animation
  - EEPROM state persistence to toggle messaging on/off via reset button
- **Message:** Configurable string (default: „IN“)

Hardware Architecture

Arduino Nano (ATmega328P)

- 16 MHz clock
- 3x Hardware timers
- 1 KB EEPROM
- 32 KB Flash memory

Time Encoding

- **Dot**: 500 ms
- **Dash**: 1500 ms (3x dot duration)
- **Intra-character gap**: 500 ms
- **Inter-character gap**: 1500 ms
- **Word gap**: 3500 ms (7x dot duration)
- **Example**: „IN“

. . - .

(2 dots)(gap)(1 dash, 1 dot)

WS2812B LED Strip

- 8 addressable LEDs
- 24-bit RGB color
- 800 kHz data rate
- Connected to Pin 6

Setup

```
5  #include <Arduino.h>
6  #include <Adafruit_NeoPixel.h>     // LED
7  #include <EEPROM.h>                // State persistence (button press
       handling)
8  // Message
9  const char MESSAGE[] = "IN";       ///< change this to whatever message you
       want
10
11 // Pin setup
12 const uint8_t LED_PIN = 6;      ///< (Digital) Pin of LED stripe
13 const uint8_t WAVE_WIDTH = 5;   ///< set to 0 for normal blinking, >0 for
       lighthouse effect
14 const uint8_t LED_COUNT = 8;    ///< total number of LEDs in the strip
15
16 // Morse Code Timing Configuration (in milliseconds)
17 const uint16_t TIME_DOT = 500;           ///< Duration of a dot
18 const uint16_t TIME_DASH = 1500;         ///< Duration of a dash
19 const uint16_t TIME_SYMBOL_GAP = 500;    ///< Gap between dots/dashes within
       a character
20 const uint16_t TIME_LETTER_GAP = 1500;  ///< Gap between characters
21 const uint16_t TIME_WORD_GAP = 3500;    ///< Gap between words
22 const uint8_t BRIGHTNESS = 1;           ///< Brightness in % from 0 to 100%,
        currently 1% very dim
23
24 // Init an instance called led_strip of Adafluit_NeoPixel class
25 Adafruit_NeoPixel led_strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
26
27 // Timing in seconds, scaled to Timer1 ticks each ISR reload
28 const uint16_t TICKS_PER_SECOND = 15625;  ///< 16MHz / 1024 = 15625
29 const uint8_t MAX_STEPS = 100;            ///< Sufficient for most messages
```

Character Encoding: Morse Lookup Table

```
47 const char* morseCode(char c) {
48   switch (c) {
49     case 'A': return ".-";
50     case 'B': return "-...";
51     case 'C': return "-.-.";
52     case 'D': return "-..";
53     case 'E': return ".";
54     case 'F': return "..-.";
55     case 'G': return "--.";
56     case 'H': return "....";
57     case 'I': return "..";
58     case 'J': return ".---";
59     case 'K': return "-.-";
60     case 'L': return ".-..";
61     case 'M': return "--";
62     case 'N': return "-.";
63     case 'O': return "---";
64     case 'P': return ".--.";
65     case 'Q': return "--.-";
66     case 'R': return ".-.";
67     case 'S': return "...";
68     case 'T': return "-";
69     case 'U': return "..-";
70     case 'V': return "...-";
71     case 'W': return ".--";
72     case 'X': return "-..-";
73     case 'Y': return "-.--";
74     case 'Z': return "--..";
75     case '0': return "-----";
76     case '1': return ".----";
77     case '2': return "..---";
78     case '3': return "...--";
79     case '4': return "....-";
80     case '5': return ".....";
81     case '6': return "-....";
82     case '7': return "--...";
83     case '8': return "---..";
84     case '9': return "----.";
85     default:  return "";
86   }
87 }
```

Building the Sequence (1/2)

Data Structures:

```
// arrays to store message
uint16_t durations[MAX_STEPS];            ///< duration in milliseconds per step
uint32_t colors[MAX_STEPS];               ///< RGB color for each step (packed as 0xRRGGBB)
uint8_t ledIndex[MAX_STEPS];              ///< Index of center LED at each time step in [0;LED_COUNT] with 255 = all LEDs
uint8_t stepCount = 0;                    ///< Total number of steps in the sequence
volatile uint8_t currentStep = 0;         ///< Current step index being executed
```

Add Step Function:

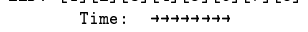
```
void addStep(uint16_t duration_ms, uint32_t color, uint8_t ledIdx = 255) {
  if (stepCount < MAX_STEPS) {
    durations[stepCount] = duration_ms;
    colors[stepCount] = color;
    ledIndex[stepCount] = ledIdx;
    stepCount++;
  }
}
```

Building the Sequence (2/2)

```
115 void buildSequence() {
116   stepCount = 0;
117   const uint32_t WHITE = led_strip.Color(255, 255, 255);
118   const uint32_t GREEN = led_strip.Color(0, 255, 0);  // not used right now
119   const uint32_t RED = led_strip.Color(255, 0, 0);    // not used right now
120   const uint32_t OFF = 0;
121
122   // loop through the message
123   for (const char* p = MESSAGE; *p; ++p) {
124     char c = toupper(*p);
125     if (c == ' ') {
126       addStep(TIME_WORD_GAP, OFF); // space between words
127       continue;
128     }
129     const char* code = morseCode(c);
130     if (!*code) continue;  // skip unknown characters
131
132     // go through each dot or dash
133     for (const char* s = code; *s; ++s) {
134       uint16_t onDur = (*s == '.') ? TIME_DOT : TIME_DASH;
135       uint16_t perLedDur = onDur / LED_COUNT;
136
137       // lighthouse wave - light moves across the strip
138       for (uint8_t i = 0; i < LED_COUNT; i++) {
139         addStep(perLedDur, WHITE, i);
140       }
141
142       if (*(s + 1)) {
143         addStep(TIME_SYMBOL_GAP, OFF);  // gap between dots/dashes
144       }
145     }
146     addStep(TIME_LETTER_GAP, OFF);  // gap between letters
147   }
148   // End of message
149   addStep(TIME_WORD_GAP, OFF);
150
151   // cool reverse wave at the end before looping
152   for (uint8_t i = LED_COUNT; i > 0; i--) {
153     addStep(20, WHITE, i);
154   }
155   addStep(TIME_WORD_GAP, OFF);
156
157 }
```

Lighthouse Wave Effect

- **Concept**: Light sweeps across LED strip instead of all-on
- **Implementation**:
  - Divide dot/dash duration by number of LEDs (8)
  - Illuminate each LED sequentially
  - Create smooth sweeping motion
- **Example**: 500 ms dot $\div$ 8 LEDs = 62.5 ms per LED
- **Wave width**: Configurable (5 LEDs default)
- **Boring**: Standard blinking is possible as well, by setting parameter $WAVE\_WIDTH = 0$



Hardware Timer1 Setup

- **Mode:** CTC (Clear Timer on Compare Match)
- **Prescaler:** 1024
- **Frequency:** 16 MHz / 1024 = 15625 Hz
- **Resolution:** 64 μs per tick

```
262   if (state) {
263     // Start messaging
264     showStartupSequence();
265
266     currentStep = 0;
267     buildSequence();
268
269     cli();  // disable interrupts
270     TCCR1A = 0;
271     TCCR1B = 0;
272     TCCR1B |= (1 << WGM12);               // CTC mode
273     TCCR1B |= (1 << CS12) | (1 << CS10); // prescaler 1024
274     OCR1A = 1562;                         // ~100ms
275     TIMSK1 |= (1 << OCIE1A);              // enable interrupt
276     sei();  // enable interrupts
277   } else {
278     // Stop messaging
279     showShutdownSequence();
280
281     // Disable Timer1 interrupt
282     TIMSK1 &= ~(1 << OCIE1A);
283     TCCR1B = 0;  // Stop timer
284   }
```

- **Calculation:** ticks = (duration_ms $\times$ 15625) / 1000

Interrupt Service Routine (ISR)

```
169 ISR(TIMER1_COMPA_vect) {
170   // Update LED state - either individual LED or all LEDs
171   if (ledIndex[currentStep] == 255 || WAVE_WIDTH == 0) {
172     // Light all LEDs (standard mode or wave disabled)
173     led_strip.fill(colors[currentStep]);
174   } else {
175     // wave effect with multiple neighbouring LEDs
176     led_strip.clear();
177     if (colors[currentStep] != 0) {
178       // Light up center LED and neighbours
179       int8_t center = ledIndex[currentStep];
180       int8_t halfWidth = WAVE_WIDTH / 2;
181
182       for (int8_t offset = -halfWidth; offset <= halfWidth; offset++) {
183         int8_t ledPos = center + offset;
184         // Only light up LEDs within strip boundaries -> else unexpected behaviour with leds on at wrong "end" of strip (like index was
                taken modulo by number of leds in strip)
185         if (ledPos >= 0 && ledPos < LED_COUNT) {
186           led_strip.setPixelColor(ledPos, colors[currentStep]);
187         }
188       }
189     }
190   }
191   led_strip.show();
192
193   // Calculate how many timer ticks for current step duration
194   uint32_t ticks = ((uint32_t)durations[currentStep] * TICKS_PER_SECOND) / 1000UL;
195   if (ticks < 100) ticks = 100;  // minimum value
196
197   OCR1A = (uint16_t)(ticks - 1);
198
199   currentStep++;
200
201   // loop back to start
202   if (currentStep >= stepCount) {
203     currentStep = 0;
204   }
205 }
```

ISR Execution Details

Responsibilities:

1. Update LED colors and positions
2. Handle wave effect logic with boundary checking
3. Transmit data to WS2812B strip (`show()`)
4. Calculate next timer interval
5. Advance to next step (with wraparound)

Timing Considerations:

- WS2812B update: $\sim 240\ \mu s$ for 8 LEDs
- Shortest Morse interval: 500 ms
- Overhead: 0.05% (negligible)

EEPROM State Management

Electrically-Erasable Programmable Read-Only Memory

```
253   // Read state (0 = Stopped, 1 = Playing)
254   byte state = EEPROM.read(0);
255   state = !state;  // toggle
256
257   // Actually EEPROM has ~100k write wear -> Only write if changed and thus really necessary to write
258   if (EEPROM.read(0) != state) {
259     EEPROM.write(0, state);
260   }
```

- **Purpose:** Remember on/off state across power cycles
- **Address 0:** Stores state (0=Stopped, 1=Playing)
- **Write protection:** Only write if value changes
- **Endurance:** ∼100,000 write cycles
- **User interface:** Reset button toggles state

Visual Feedback Sequences

Startup Sequence:

```
210 void showStartupSequence() {
211   led_strip.fill(led_strip.Color(255, 0, 0)); // Red
212   led_strip.show();
213   delay(500);
214   led_strip.fill(led_strip.Color(0, 255, 0)); // Green
215   led_strip.show();
216   delay(500);
217   led_strip.fill(0); // Off
218   led_strip.show();
219   delay(500);
220 }
```

Shutdown Sequence:

```
225 void showShutdownSequence() {
226   led_strip.fill(led_strip.Color(0, 255, 0)); // Green
227   led_strip.show();
228   delay(500);
229   led_strip.fill(led_strip.Color(255, 0, 0)); // Red
230   led_strip.show();
231   delay(500);
232   led_strip.fill(0); // Off
233   led_strip.show();
234 }
```

Startup: Red $\rightarrow$ Green $\rightarrow$ Off $\rightarrow$ Transmit
Shutdown: Green $\rightarrow$ Red $\rightarrow$ Off

Conclusion

- **Successfully implemented** interrupt-driven Morse code transmitter with lighthouse effect

```
292 void loop() {
293   // main loop free for other tasks
294 }
```

- **Demonstrated concepts:**
  - Hardware timer configuration (Timer1 CTC mode)
  - Interrupt service routines (ISR)
  - Non-volatile memory (EEPROM)
  - Addressable LED control (WS2812B protocol)

Thank you for your attention!

Questions?

Appendix: Full Code - Part 1

```
0  // Arduino: Nano Atmel mega328p
1  // LED: WS2812B (CJMCU-2812-8)
2  // Used: Timer1 CTC, prescaler 1024, Button interupt
3
4  #include <Arduino.h>
5  #include <Adafruit_NeoPixel.h>     // LED
6  #include <EEPROM.h>                // State persistence (button press handling)
7  // Message
8  const char MESSAGE[] = "IN";       ///< change this to whatever message you want
9
10 // Pin setup
11 const uint8_t LED_PIN = 6;      ///< (Digital) Pin of LED stripe
12 const uint8_t WAVE_WIDTH = 5;   ///< set to 0 for normal blinking, >0 for lighthouse effect
13 const uint8_t LED_COUNT = 8;    ///< total number of LEDs in the strip
14
15 // Morse Code Timing Configuration (in milliseconds)
16 const uint16_t TIME_DOT = 500;          ///< Duration of a dot
17 const uint16_t TIME_DASH = 1500;        ///< Duration of a dash
18 const uint16_t TIME_SYMBOL_GAP = 500;   ///< Gap between dots/dashes within a character
19 const uint16_t TIME_LETTER_GAP = 1500;  ///< Gap between characters
20 const uint16_t TIME_WORD_GAP = 3500;    ///< Gap between words
21 const uint8_t BRIGHTNESS = 1;           ///< Brightness in % from 0 to 100%, currently 1% very dim
22
23 // Init an instance called led_strip of Adafruit_NeoPixel class
24 Adafruit_NeoPixel led_strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
25
26 // Timing in seconds, scaled to Timer1 ticks each ISR reload
27 const uint16_t TICKS_PER_SECOND = 15625;  ///< 16MHz / 1024 = 15625
28 const uint8_t MAX_STEPS = 100;            ///< Sufficient for most messages
29
30 // arrays to store message
31 uint16_t durations[MAX_STEPS];            ///< duration in milliseconds per step
32 uint32_t colors[MAX_STEPS];               ///< RGB color for each step (packed as 0xRRGGBB)
33 uint8_t ledIndex[MAX_STEPS];              ///< Index of center LED at each time step in [0;LED_COUNT] with 255 = all LEDs
34 uint8_t stepCount = 0;                    ///< Total number of steps in the sequence
35 volatile uint8_t currentStep = 0;         ///< Current step index being executed
36
37 /**
38  * @brief Lookup table for Morse symbols.
39  *
```

Appendix: Full Code - Part 2

```
41  * Converts a single character into its Morse code representation
42  * using dots ('.') and dashes ('-').
43  *
44  * @param c The character to convert (case-insensitive handling done by caller).
45  * @return const char* String representing the Morse code, or empty string if unknown.
46  */
47 const char* morseCode(char c) {
48   switch (c) {
49     case 'A': return ".-";
50     case 'B': return "-...";
51     case 'C': return "-.-.";
52     case 'D': return "-..";
53     case 'E': return ".";
54     case 'F': return "..-.";
55     case 'G': return "--.";
56     case 'H': return "....";
57     case 'I': return "..";
58     case 'J': return ".---";
59     case 'K': return "-.-";
60     case 'L': return ".-..";
61     case 'M': return "--";
62     case 'N': return "-.";
63     case 'O': return "---";
64     case 'P': return ".--.";
65     case 'Q': return "--.-";
66     case 'R': return ".-.";
67     case 'S': return "...";
68     case 'T': return "-";
69     case 'U': return "..-";
70     case 'V': return "...-";
71     case 'W': return ".--";
72     case 'X': return "-..-";
73     case 'Y': return "-.--";
74     case 'Z': return "--..";
75     case '0': return "-----";
76     case '1': return ".----";
77     case '2': return "..---";
78     case '3': return "...--";
79     case '4': return "....-";
80     case '5': return ".....";
81     case '6': return "-....";
82     case '7': return "--...";
83     case '8': return "---..";
84     case '9': return "----.";
85     default:  return "";
86   }
87 }
```

Appendix: Full Code - Part 3

```
89 /**
90  * @brief Adds a single step to the Morse sequence.
91  *
92  * Records the duration and color for a specific part of the sequence.
93  *
94  * @param duration_ms Duration of the step in milliseconds.
95  * @param color RGB color (use led_strip.Color() or 0 for off).
96  * @param ledIdx LED index to light (255 = all LEDs).
97  */
98 void addStep(uint16_t duration_ms, uint32_t color, uint8_t ledIdx = 255) {
99   if (stepCount < MAX_STEPS) {
100    durations[stepCount] = duration_ms;
101    colors[stepCount] = color;
102    ledIndex[stepCount] = ledIdx;
103    stepCount++;
104  }
105 }
106
107 /**
108  * @brief Constructs the full Morse code sequence for the global MESSAGE.
109  *
110  * Converts each character to Morse code and creates a lighthouse wave effect
111  * where the light sweeps across all LEDs for each dot/dash.
112  * Adds appropriate gaps between symbols, letters, and words.
113  * Includes a final wave effect at the end before looping.
114  */
115 void buildSequence() {
116   stepCount = 0;
117   const uint32_t WHITE = led_strip.Color(255, 255, 255);
118   const uint32_t GREEN = led_strip.Color(0, 255, 0);  // not used right now
119   const uint32_t RED = led_strip.Color(255, 0, 0);    // not used right now
120   const uint32_t OFF = 0;
121
122   // loop through the message
123   for (const char* p = MESSAGE; *p; ++p) {
124     char c = toupper(*p);
125     if (c == ' ') {
126       addStep(TIME_WORD_GAP, OFF); // space between words
127       continue;
128     }
129     const char* code = morseCode(c);
130     if (!*code) continue;  // skip unknown characters
```

Appendix: Full Code - Part 4

```
    // go through each dot or dash
    for (const char* s = code; *s; ++s) {
      uint16_t onDur = (*s == '.') ? TIME_DOT : TIME_DASH;
      uint16_t perLedDur = onDur / LED_COUNT;

      // lighthouse wave - light moves across the strip
      for (uint8_t i = 0; i < LED_COUNT; i++) {
        addStep(perLedDur, WHITE, i);
      }

      if (*(s + 1)) {
        addStep(TIME_SYMBOL_GAP, OFF);  // gap between dots/dashes
      }
    }
    addStep(TIME_LETTER_GAP, OFF);  // gap between letters
  }
  // End of message
  addStep(TIME_WORD_GAP, OFF);

  // cool reverse wave at the end before looping
  for (uint8_t i = LED_COUNT; i > 0; i--) {
    addStep(20, WHITE, i);
  }
  addStep(TIME_WORD_GAP, OFF);

}

/**
 * @brief Timer1 Compare Match A Interrupt Service Routine.
 *
 * Updates the LED state for the current step in the sequence.
 * Handles both full strip control and wave effects with multiple LEDs.
 * The sequence loops automatically when complete.
 *
 * Note: LED updates can block for ~300 microseconds (WS2812B protocol).
 * This is acceptable given the slow Morse code timing (500ms+).
 */
```

Appendix: Full Code - Part 5

```
169 ISR(TIMER1_COMPA_vect) {
170   // Update LED state - either individual LED or all LEDs
171   if (ledIndex[currentStep] == 255 || WAVE_WIDTH == 0) {
172     // Light all LEDs (standard mode or wave disabled)
173     led_strip.fill(colors[currentStep]);
174   } else {
175     // wave effect with multiple neighbouring LEDs
176     led_strip.clear();
177     if (colors[currentStep] != 0) {
178       // Light up center LED and neighbours
179       int8_t center = ledIndex[currentStep];
180       int8_t halfWidth = WAVE_WIDTH / 2;
181
182       for (int8_t offset = -halfWidth; offset <= halfWidth; offset++) {
183         int8_t ledPos = center + offset;
184         // Only light up LEDs within strip boundaries -> else unexpected behaviour with leds on at wrong "end" of strip (like index was
                taken modulo by number of leds in strip)
185         if (ledPos >= 0 && ledPos < LED_COUNT) {
186           led_strip.setPixelColor(ledPos, colors[currentStep]);
187         }
188       }
189     }
190   }
191   led_strip.show();
192
193   // Calculate how many timer ticks for current step duration
194   uint32_t ticks = ((uint32_t)durations[currentStep] * TICKS_PER_SECOND) / 1000UL;
195   if (ticks < 100) ticks = 100;  // minimum value
196
197   OCR1A = (uint16_t)(ticks - 1);
198
199   currentStep++;
200
201   // loop back to start
202   if (currentStep >= stepCount) {
203     currentStep = 0;
204   }
205 }
```

Appendix: Full Code - Part 6

```
/**
 * @brief Displays startup sequence: Red -> Green -> Off.
 */
void showStartupSequence() {
  led_strip.fill(led_strip.Color(255, 0, 0)); // Red
  led_strip.show();
  delay(500);
  led_strip.fill(led_strip.Color(0, 255, 0)); // Green
  led_strip.show();
  delay(500);
  led_strip.fill(0); // Off
  led_strip.show();
  delay(500);
}

/**
 * @brief Displays shutdown sequence: Green -> Red -> Off.
 */
void showShutdownSequence() {
  led_strip.fill(led_strip.Color(0, 255, 0)); // Green
  led_strip.show();
  delay(500);
  led_strip.fill(led_strip.Color(255, 0, 0)); // Red
  led_strip.show();
  delay(500);
  led_strip.fill(0); // Off
  led_strip.show();
}

/**
 * @brief Arduino setup function.
 *
 * Toggles between playing and stopped states using EEPROM.
 * Reset button press during play: restarts from beginning
 * Reset button press when stopped: starts playing
 */
void setup() {
  led_strip.begin();

  // Calculate and set brightness
  uint8_t brightness = (255 * BRIGHTNESS) / 100;
  led_strip.setBrightness(brightness);
```

Appendix: Full Code - Part 7

```
250   // Using EEPROM to remember state between resets
251   // Pressing reset button toggles on / off
252
253   // Read state (0 = Stopped, 1 = Playing)
254   byte state = EEPROM.read(0);
255   state = !state;  // toggle
256
257   // Actually EEPROM has ~100k write wear -> Only write if changed and thus really necessary to write
258   if (EEPROM.read(0) != state) {
259     EEPROM.write(0, state);
260   }
261
262   if (state) {
263     // Start messaging
264     showStartupSequence();
265
266     currentStep = 0;
267     buildSequence();
268
269     cli();  // disable interrupts
270     TCCR1A = 0;
271     TCCR1B = 0;
272     TCCR1B |= (1 << WGM12);               // CTC mode
273     TCCR1B |= (1 << CS12) | (1 << CS10); // prescaler 1024
274     OCR1A = 1562;                         // ~100ms
275     TIMSK1 |= (1 << OCIE1A);              // enable interrupt
276     sei();  // enable interrupts
277   } else {
278     // Stop messaging
279     showShutdownSequence();
280
281     // Disable Timer1 interrupt
282     TIMSK1 &= ~(1 << OCIE1A);
283     TCCR1B = 0;  // Stop timer
284   }
285 }
```

Appendix: Full Code - Part 8

```
287 /**
288  * @brief Main application loop.
289  *
290  * Empty - all Morse code logic is interrupt-driven via Timer1.
291  */
292 void loop() {
293   // main loop free for other tasks
294 }
```